Below are **(1) JSON structures** + **(2) JavaScript generator mappings** (DOCX) for templates **1, 3, 6, 7**, following the same pattern as your reference (`getDocx()`, `docxColors()`, `safeStr()`, tables with `hdrCell/dataCell/statusCell`, returns `{ blob, filename }`).

---

1) Incident Resolution Summary (Customer-Friendly)

JSON structure

```
{
  "incident_number": "INC0012345",
  "short_description": "Users unable to login to portal",
  "priority": "P2",
  "service": "Customer Portal",
  "assignment_group": "Service Desk",
  "opened_at": "2026-04-20 09:12",
  "resolved_at": "2026-04-20 11:05",
  "created_date": "2026-04-25",
  "author": "Your Name",

  "impact": {
    "impact_start": "2026-04-20 09:05",
    "impact_end": "2026-04-20 11:05",
    "affected_users": "All external users",
    "business_impact": "Login failures, users unable to submit requests",
    "customer_visible": "Yes"
  },

  "symptoms": [
    "Login redirects to error page",
    "SSO callback returns 500"
  ],

  "resolution": {
    "summary": "Restarted identity provider pods and cleared stale cache",
    "workaround": "Use direct app URL (bypasses portal) until fix applied",
    "root_cause_if_known": "Expired signing certificate in IdP cache"
  },

  "resolution_steps": [
    { "step": "Validated error in IdP logs", "owner": "Platform Team" },
    { "step": "Restarted IdP pods", "owner": "Platform Team" },
    { "step": "Verified login success with test users", "owner": "Service Desk" }
  ],

  "next_steps": [
    { "action": "Implement cert rotation monitoring", "owner": "IAM Team",
"due_date": "2026-05-05", "status": "Planned" }
  ],

  "related_records": {
```

```
    "problem": "PRB0010203",
    "change": "CHG0032109",
    "kb": "KB0010099"
  },

  "stakeholders": [
    { "name": "Customer IT Lead", "role": "Approver/Stakeholder", "contact":
"lead@customer.com" }
  ]
}
```

---

3) Post-Incident Review (PIR) / RCA Report

JSON structure

```
{
  "incident_number": "INC0012345",
  "major_incident": "MI0000456",
  "service": "Customer Portal",
  "priority": "P1",
  "created_date": "2026-04-25",
  "author": "Your Name",
  "executive_summary": "Authentication outage caused by stale IdP cert cache;
resolved by restart and cache clear.",

  "impact": {
    "start": "2026-04-20 09:05",
    "end": "2026-04-20 11:05",
    "duration_minutes": 120,
    "user_impact": "100% external users impacted",
    "business_impact": "Requests could not be submitted"
  },

  "detection_and_response": {
    "detection_method": "User reports + monitoring alert",
    "mttd_minutes": 5,
    "mttr_minutes": 120,
    "escalation_path": "Service Desk -> IAM -> Platform"
  },

  "timeline": [
    { "time": "09:05", "event": "First failures observed", "owner": "Monitoring"
},
    { "time": "09:12", "event": "Incident opened", "owner": "Service Desk" },
    { "time": "10:20", "event": "Root cause suspected in IdP cache", "owner":
"IAM" },
    { "time": "11:05", "event": "Service restored", "owner": "Platform" }
  ],

  "root_cause_analysis": {
```

```
      "method": "5 Whys",
      "primary_cause": "Expired signing certificate persisted in IdP cache",
      "contributing_factors": [
        "No proactive cert expiry alerting",
        "Restart runbook not documented"
      ]
    },

    "what_went_well": [
      "Quick escalation to Platform team",
      "Clear stakeholder updates after MI declared"
    ],
    "what_didnt_go_well": [
      "Monitoring alert was noisy and not specific",
      "No automated cert rotation validation"
    ],

    "corrective_actions": [
      { "action": "Add cert expiry monitoring & alerting", "type": "Preventive",
"owner": "IAM", "due_date": "2026-05-05", "status": "Planned" },
      { "action": "Create IdP restart runbook", "type": "Process", "owner":
"Platform", "due_date": "2026-04-30", "status": "In Progress" }
    ],

    "evidence_links": [
      { "label": "IdP Logs", "url": "https://example/logs/123" },
      { "label": "Monitoring Alert", "url": "https://example/alerts/456" }
    ]
  }
```

---

6) Deployment Runbook (Ops Execution)

JSON structure

```
  {
    "runbook_title": "Deploy Portal Patch v1.2.8",
    "service": "Customer Portal",
    "environment": "PROD",
    "version": "1.0",
    "created_date": "2026-04-25",
    "author": "Your Name",
    "change_request": "CHG0032109",
    "maintenance_window": "2026-04-28 22:00-23:00 UTC",

    "prerequisites": [
      "Approved change in Scheduled state",
      "Backup completed successfully",
      "Access to Kubernetes cluster + admin role"
    ],
    "access_requirements": [
```

```
      "VPN connected",
      "kubectl configured",
      "Elevated role for deployment namespace"
    ],

    "pre_checks": [
      "Confirm current version running",
      "Confirm error budget/health green",
      "Confirm monitoring dashboard available"
    ],

    "execution_steps": [
      {
        "step_no": 1,
        "action": "Scale portal replicas to 0 (if required)",
        "command": "kubectl scale deploy portal --replicas=0 -n prod",
        "expected_result": "Pods terminated cleanly",
        "validation": "kubectl get pods -n prod shows 0 portal pods",
        "stop_go": "GO if no errors; STOP if termination hangs >5 mins"
      }
    ],

    "post_checks": [
      "Login test with test user",
      "Submit sample request end-to-end",
      "Check error rate < 1% for 15 mins"
    ],

    "monitoring": [
      "Dashboard: Portal Latency",
      "Dashboard: Auth Success Rate"
    ],

    "rollback_steps": [
      { "step_no": 1, "action": "Rollback to previous image tag", "command":
  "kubectl rollout undo deploy/portal -n prod", "validation": "Version reverted and
  login succeeds" }
    ],

    "contacts": [
      { "name": "On-call Platform", "role": "Executor", "contact": "+1-xxx" },
      { "name": "Service Owner", "role": "Approver", "contact": "owner@company.com"
  }
    ]
  }
```

---

7) ServiceNow Enhancement / User Story Technical Design

JSON structure

```
{
  "story_id": "STRY0010091",
  "title": "Auto-route incidents based on CI + Service",
  "requested_by": "Customer IT",
  "product_owner": "PO Name",
  "created_date": "2026-04-25",
  "author": "Your Name",

  "summary": "Implement routing logic using CI/service mapping to reduce
assignment delays.",
  "in_scope": [
    "Flow/BR for routing",
    "Update assignment group mapping table",
    "Unit + UAT support"
  ],
  "out_of_scope": [
    "Redesign of incident form UI"
  ],
  "assumptions": [
    "CMDB relationships are accurate for targeted services"
  ],

  "acceptance_criteria": [
    "Incidents with CI populated are routed within 30 seconds",
    "Fallback rule applies if CI missing"
  ],

  "design": {
    "overview": "Use Flow Designer for routing; fallback to BR for edge cases."
  },

  "data_model_changes": [
    { "table": "u_ci_service_map", "change": "New table", "details": "Maps CI
class + service to assignment group" },
    { "table": "incident", "change": "Field update", "details": "Ensure business
rule reads cmdb_ci + business_service" }
  ],

  "implementation_components": [
    { "type": "Flow", "name": "INC Auto Route", "description": "Routes on
create/update when CI changes" },
    { "type": "Business Rule", "name": "INC Route Fallback", "description": "Runs
if flow cannot determine group" }
  ],

  "security": [
    { "item": "ACL", "details": "Only ITIL can edit mapping table" },
    { "item": "Role", "details": "Create role x_app.mapping_admin" }
  ],

  "integrations": [
    { "system": "None", "details": "No external integration for this story" }
```

```
  ],

  "testing": {
    "test_cases": [
      { "id": "TC01", "scenario": "CI+Service match", "expected": "Assignment
group set to mapped group" },
      { "id": "TC02", "scenario": "CI missing", "expected": "Fallback group used"
}
    ]
  },

  "deployment_notes": [
    "Update set naming: STRY0010091_Routing",
    "Promote to TEST then PROD after UAT sign-off"
  ],

  "risks": [
    { "description": "Bad CMDB data may misroute incidents", "impact": "High",
"mitigation": "Limit to 3 services initially + monitor" }
  ],

  "open_questions": [
    "Which services are in phase 1 rollout?"
  ]
}
```

JavaScript mapping (DOCX generators)

> Add these to your `main.js` after existing generators. Assumes you already have: `getDocx()`, `docxColors()`, `safeStr()` like your reference.

```
// ── Shared Helpers (re-use across templates) ──────────────────────────
function makeDocHelpers() {
  const { Paragraph, TextRun, TableCell, AlignmentType, BorderStyle, WidthType,
ShadingType } = getDocx();
  const C = docxColors();

  const bdr = { style: BorderStyle.SINGLE, size: 1, color: C.MGRAY };
  const borders = { top: bdr, bottom: bdr, left: bdr, right: bdr };

  function para(t, bold, color, size) {
    return new Paragraph({
      spacing: { before: 60, after: 60 },
      children: [new TextRun({ text: safeStr(t), bold: !!bold, color: color ||
C.GRAY, size: size || 20, font: 'Arial' })]
    });
  }

  function h1(t) {
```

```
    return new Paragraph({
      spacing: { before: 120, after: 120 },
      children: [new TextRun({ text: safeStr(t), bold: true, color: C.NAVY, size:
32, font: 'Arial' })]
    });
  }

  function h2(t) {
    return new Paragraph({
      spacing: { before: 240, after: 120 },
      children: [new TextRun({ text: safeStr(t), bold: true, color: C.BLUE, size:
28, font: 'Arial' })]
    });
  }

  function bullets(arr) {
    const items = Array.isArray(arr) ? arr : [];
    return items
      .filter(x => safeStr(x).trim().length > 0)
      .map(x => new Paragraph({
        spacing: { before: 20, after: 20 },
        bullet: { level: 0 },
        children: [new TextRun({ text: safeStr(x), color: C.GRAY, size: 20, font:
'Arial' })]
      }));
  }

  function hdrCell(t, w) {
    return new TableCell({
      borders,
      width: { size: w, type: WidthType.DXA },
      shading: { fill: C.BLUE, type: ShadingType.CLEAR },
      margins: { top: 80, bottom: 80, left: 120, right: 120 },
      children: [new Paragraph({ children: [new TextRun({ text: safeStr(t), bold:
true, color: C.WHITE, size: 18, font: 'Arial' })] })]
    });
  }

  function dataCell(t, w, bg) {
    return new TableCell({
      borders,
      width: { size: w, type: WidthType.DXA },
      shading: { fill: bg || C.WHITE, type: ShadingType.CLEAR },
      margins: { top: 70, bottom: 70, left: 120, right: 120 },
      children: [new Paragraph({ children: [new TextRun({ text: safeStr(t), color:
C.GRAY, size: 18, font: 'Arial' })] })]
    });
  }

  function statusCell(t, w) {
    const s = (t || '').toLowerCase();
    let bg, tc;
    if (s.includes('completed') || s.includes('done') || s.includes('on track')) {
bg = C.LIGHT_GREEN; tc = C.GREEN; }
```

```
    else if (s.includes('in progress') || s.includes('ongoing') ||
s.includes('pending')) { bg = C.LIGHT_BLUE; tc = C.BLUE; }
    else if (s.includes('behind') || s.includes('delayed') ||
s.includes('blocked')) { bg = C.LIGHT_AMBER; tc = C.AMBER; }
    else { bg = C.LIGHT_GRAY; tc = C.GRAY; }

    return new TableCell({
      borders,
      width: { size: w, type: WidthType.DXA },
      shading: { fill: bg, type: ShadingType.CLEAR },
      margins: { top: 70, bottom: 70, left: 120, right: 120 },
      children: [new Paragraph({ children: [new TextRun({ text: safeStr(t), bold:
true, color: tc, size: 18, font: 'Arial' })] })]
    });
  }

  return { C, para, h1, h2, bullets, hdrCell, dataCell, statusCell };
}


// ── (1) Incident Resolution Summary ──────────────────────────────────────
async function generateIncidentResolutionSummary(data) {
  const { Document, Packer, Table, TableRow, WidthType } = getDocx();
  const { C, para, h1, h2, bullets, hdrCell, dataCell, statusCell } =
makeDocHelpers();

  const inc = safeStr(data.incident_number || 'INC');
  const title = inc + ' - Incident Resolution Summary';
  const createdDate = safeStr(data.created_date || new
Date().toLocaleDateString());
  const author = safeStr(data.author || 'Author');

  const impact = data.impact || {};
  const symptoms = data.symptoms || [];
  const res = data.resolution || {};
  const resSteps = data.resolution_steps || [];
  const nextSteps = data.next_steps || [];
  const rel = data.related_records || {};

  const children = [
    h1(title),
    para(`Created: ${createdDate}`, false, C.GRAY, 18),
    para(`Author: ${author}`, false, C.GRAY, 18),
    para(''),

    h2('Incident Details')
  ];

  const detailsTable = new Table({
    width: { size: 9360, type: WidthType.DXA },
    columnWidths: [2500, 6860],
    rows: [
      new TableRow({ children: [hdrCell('Field', 2500), hdrCell('Value', 6860)]
}),
```

```
      new TableRow({ children: [dataCell('Short Description', 2500),
dataCell(data.short_description || '', 6860)] }),
      new TableRow({ children: [dataCell('Priority', 2500),
statusCell(data.priority || '', 6860)] }),
      new TableRow({ children: [dataCell('Service', 2500), dataCell(data.service
|| '', 6860)] }),
      new TableRow({ children: [dataCell('Assignment Group', 2500),
dataCell(data.assignment_group || '', 6860)] }),
      new TableRow({ children: [dataCell('Opened At', 2500),
dataCell(data.opened_at || '', 6860)] }),
      new TableRow({ children: [dataCell('Resolved At', 2500),
dataCell(data.resolved_at || '', 6860)] })
    ]
  });
  children.push(detailsTable, para(''));

  children.push(h2('Impact'));
  children.push(...bullets([
    `Impact Window: ${safeStr(impact.impact_start)} →
${safeStr(impact.impact_end)}`.trim(),
    `Affected Users: ${safeStr(impact.affected_users)}`,
    `Customer Visible: ${safeStr(impact.customer_visible)}`,
    `Business Impact: ${safeStr(impact.business_impact)}`
  ]));

  if (symptoms.length) {
    children.push(para(''), h2('Symptoms Observed'), ...bullets(symptoms));
  }

  children.push(para(''), h2('Resolution'));
  children.push(...bullets([
    `Summary: ${safeStr(res.summary)}`,
    res.root_cause_if_known ? `Root Cause (if known):
${safeStr(res.root_cause_if_known)}` : '',
    res.workaround ? `Workaround: ${safeStr(res.workaround)}` : ''
  ].filter(Boolean)));

  if (resSteps.length) {
    children.push(para(''), h2('Resolution Steps'));
    const stepsTable = new Table({
      width: { size: 9360, type: WidthType.DXA },
      columnWidths: [800, 6160, 2400],
      rows: [
        new TableRow({ children: [hdrCell('#', 800), hdrCell('Step', 6160),
hdrCell('Owner', 2400)] }),
        ...resSteps.map((s, i) => new TableRow({
          children: [
            dataCell(String(i + 1), 800, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(s.step || '', 6160, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(s.owner || '', 2400, i % 2 ? C.LIGHT_GRAY : C.WHITE)
          ]
        }))
      ]
    });
```

```
    children.push(stepsTable);
  }

  if (nextSteps.length) {
    children.push(para(''), h2('Next Steps / Actions'));
    const actionsTable = new Table({
      width: { size: 9360, type: WidthType.DXA },
      columnWidths: [5000, 1800, 1400, 1160],
      rows: [
        new TableRow({ children: [hdrCell('Action', 5000), hdrCell('Owner', 1800),
hdrCell('Due Date', 1400), hdrCell('Status', 1160)] }),
        ...nextSteps.map((a, i) => new TableRow({
          children: [
            dataCell(a.action || '', 5000, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(a.owner || '', 1800, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(a.due_date || '', 1400, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            statusCell(a.status || '', 1160)
          ]
        }))
      ]
    });
    children.push(actionsTable);
  }

  children.push(para(''), h2('Related Records'));
  children.push(...bullets([
    rel.problem ? `Problem: ${safeStr(rel.problem)}` : '',
    rel.change ? `Change: ${safeStr(rel.change)}` : '',
    rel.kb ? `Knowledge: ${safeStr(rel.kb)}` : ''
  ].filter(Boolean)));

  const doc = new Document({ sections: [{ properties: {}, children }] });
  const blob = await Packer.toBlob(doc);
  return { blob, filename: `${inc}_Incident_Resolution_Summary.docx`.replace(/[^a-
z0-9._-]/gi, '_') };
}


// ── (3) PIR / RCA Report ──────────────────────────────────────────────────
async function generatePostIncidentReviewRCA(data) {
  const { Document, Packer, Table, TableRow, WidthType } = getDocx();
  const { C, para, h1, h2, bullets, hdrCell, dataCell, statusCell } =
makeDocHelpers();

  const inc = safeStr(data.incident_number || 'INC');
  const title = `${inc} - Post Incident Review (PIR / RCA)`;
  const createdDate = safeStr(data.created_date || new
Date().toLocaleDateString());
  const author = safeStr(data.author || 'Author');

  const impact = data.impact || {};
  const dr = data.detection_and_response || {};
  const timeline = data.timeline || [];
  const rca = data.root_cause_analysis || {};
```

```
  const actions = data.corrective_actions || [];
  const links = data.evidence_links || [];

  const children = [
    h1(title),
    para(`Created: ${createdDate}`, false, C.GRAY, 18),
    para(`Author: ${author}`, false, C.GRAY, 18),
    para(''),

    h2('Executive Summary'),
    para(data.executive_summary || '')
  ];

  children.push(para(''), h2('Impact'));
  children.push(...bullets([
    `Incident / MI: ${safeStr(data.incident_number)} ${data.major_incident ? `|
${safeStr(data.major_incident)}` : ''}`.trim(),
    `Service: ${safeStr(data.service)}`,
    `Priority: ${safeStr(data.priority)}`,
    `Start → End: ${safeStr(impact.start)} → ${safeStr(impact.end)}`,
    impact.duration_minutes ? `Duration (mins):
${safeStr(impact.duration_minutes)}` : '',
    `User Impact: ${safeStr(impact.user_impact)}`,
    `Business Impact: ${safeStr(impact.business_impact)}`
  ].filter(Boolean)));

  children.push(para(''), h2('Detection & Response'));
  children.push(...bullets([
    `Detection Method: ${safeStr(dr.detection_method)}`,
    dr.mttd_minutes !== undefined ? `MTTD (mins): ${safeStr(dr.mttd_minutes)}` :
'',
    dr.mttr_minutes !== undefined ? `MTTR (mins): ${safeStr(dr.mttr_minutes)}` :
'',
    `Escalation Path: ${safeStr(dr.escalation_path)}`
  ].filter(Boolean)));

  if (timeline.length) {
    children.push(para(''), h2('Timeline'));
    const timelineTable = new Table({
      width: { size: 9360, type: WidthType.DXA },
      columnWidths: [1400, 6160, 1800],
      rows: [
        new TableRow({ children: [hdrCell('Time', 1400), hdrCell('Event', 6160),
hdrCell('Owner', 1800)] }),
        ...timeline.map((t, i) => new TableRow({
          children: [
            dataCell(t.time || '', 1400, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(t.event || '', 6160, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(t.owner || '', 1800, i % 2 ? C.LIGHT_GRAY : C.WHITE)
          ]
        }))
      ]
    });
    children.push(timelineTable);
```

```
  }

  children.push(para(''), h2('Root Cause Analysis'));
  children.push(...bullets([
    rca.method ? `Method: ${safeStr(rca.method)}` : '',
    rca.primary_cause ? `Primary Cause: ${safeStr(rca.primary_cause)}` : ''
  ].filter(Boolean)));

  if (Array.isArray(rca.contributing_factors) && rca.contributing_factors.length)
{
    children.push(para('Contributing Factors:', true));
    children.push(...bullets(rca.contributing_factors));
  }

  if (Array.isArray(data.what_went_well) && data.what_went_well.length) {
    children.push(para(''), h2('What Went Well'),
...bullets(data.what_went_well));
  }
  if (Array.isArray(data.what_didnt_go_well) && data.what_didnt_go_well.length) {
    children.push(para(''), h2('What Didn’t Go Well'),
...bullets(data.what_didnt_go_well));
  }

  if (actions.length) {
    children.push(para(''), h2('Corrective Actions'));
    const actionsTable = new Table({
      width: { size: 9360, type: WidthType.DXA },
      columnWidths: [4200, 1400, 1600, 1200, 960],
      rows: [
        new TableRow({ children: [hdrCell('Action', 4200), hdrCell('Type', 1400),
hdrCell('Owner', 1600), hdrCell('Due', 1200), hdrCell('Status', 960)] }),
        ...actions.map((a, i) => new TableRow({
          children: [
            dataCell(a.action || '', 4200, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(a.type || '', 1400, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(a.owner || '', 1600, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(a.due_date || '', 1200, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            statusCell(a.status || '', 960)
          ]
        }))
      ]
    });
    children.push(actionsTable);
  }

  if (links.length) {
    children.push(para(''), h2('Evidence / Links'));
    children.push(...bullets(links.map(l => `${safeStr(l.label)}:
${safeStr(l.url)}`)));
  }

  const doc = new Document({ sections: [{ properties: {}, children }] });
  const blob = await Packer.toBlob(doc);
  return { blob, filename: `${inc}_PIR_RCA.docx`.replace(/[^a-z0-9._-]/gi, '_') };
```

```
}


// ── (6) Deployment Runbook ──────────────────────────────────────────
async function generateDeploymentRunbook(data) {
  const { Document, Packer, Table, TableRow, WidthType } = getDocx();
  const { C, para, h1, h2, bullets, hdrCell, dataCell } = makeDocHelpers();

  const title = safeStr(data.runbook_title || 'Deployment Runbook');
  const createdDate = safeStr(data.created_date || new
Date().toLocaleDateString());
  const author = safeStr(data.author || 'Author');

  const execSteps = data.execution_steps || [];
  const rollback = data.rollback_steps || [];

  const children = [
    h1(`${title} - Runbook`),
    para(`Created: ${createdDate}`, false, C.GRAY, 18),
    para(`Author: ${author}`, false, C.GRAY, 18),
    para('')
  ];

  children.push(h2('Change / Context'));
  children.push(...bullets([
    `Service: ${safeStr(data.service)}`,
    `Environment: ${safeStr(data.environment)}`,
    `Version: ${safeStr(data.version)}`,
    `Change Request: ${safeStr(data.change_request)}`,
    `Maintenance Window: ${safeStr(data.maintenance_window)}`
  ].filter(x => !x.endsWith(': '))));

  if (Array.isArray(data.prerequisites) && data.prerequisites.length) {
    children.push(para(''), h2('Prerequisites'), ...bullets(data.prerequisites));
  }
  if (Array.isArray(data.access_requirements) && data.access_requirements.length)
{
    children.push(para(''), h2('Access Requirements'),
...bullets(data.access_requirements));
  }
  if (Array.isArray(data.pre_checks) && data.pre_checks.length) {
    children.push(para(''), h2('Pre-Checks'), ...bullets(data.pre_checks));
  }

  if (execSteps.length) {
    children.push(para(''), h2('Execution Steps'));
    const stepsTable = new Table({
      width: { size: 9360, type: WidthType.DXA },
      columnWidths: [600, 2900, 2500, 1700, 1660],
      rows: [
        new TableRow({ children: [
          hdrCell('#', 600),
          hdrCell('Action', 2900),
          hdrCell('Command / Script', 2500),
```

```
          hdrCell('Expected Result', 1700),
          hdrCell('Validation / Stop-Go', 1660)
        ]}),
        ...execSteps.map((s, i) => new TableRow({
          children: [
            dataCell(s.step_no != null ? String(s.step_no) : String(i + 1), 600, i
% 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(s.action || '', 2900, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(s.command || '', 2500, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(s.expected_result || '', 1700, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
            dataCell([s.validation, s.stop_go].filter(Boolean).join(' | '), 1660,
i % 2 ? C.LIGHT_GRAY : C.WHITE)
          ]
        }))
      ]
    });
    children.push(stepsTable);
  }

  if (Array.isArray(data.post_checks) && data.post_checks.length) {
    children.push(para(''), h2('Post-Checks'), ...bullets(data.post_checks));
  }
  if (Array.isArray(data.monitoring) && data.monitoring.length) {
    children.push(para(''), h2('Monitoring'), ...bullets(data.monitoring));
  }

  if (rollback.length) {
    children.push(para(''), h2('Rollback Steps'));
    const rbTable = new Table({
      width: { size: 9360, type: WidthType.DXA },
      columnWidths: [600, 3600, 2800, 2360],
      rows: [
        new TableRow({ children: [hdrCell('#', 600), hdrCell('Action', 3600),
hdrCell('Command', 2800), hdrCell('Validation', 2360)] }),
        ...rollback.map((s, i) => new TableRow({
          children: [
            dataCell(s.step_no != null ? String(s.step_no) : String(i + 1), 600, i
% 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(s.action || '', 3600, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(s.command || '', 2800, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(s.validation || '', 2360, i % 2 ? C.LIGHT_GRAY : C.WHITE)
          ]
        }))
      ]
    });
    children.push(rbTable);
  }

  if (Array.isArray(data.contacts) && data.contacts.length) {
    children.push(para(''), h2('Contacts'), ...bullets(data.contacts.map(c =>
`${safeStr(c.role)} - ${safeStr(c.name)} (${safeStr(c.contact)})`)));
  }
```

```
  const doc = new Document({ sections: [{ properties: {}, children }] });
  const blob = await Packer.toBlob(doc);
  return { blob, filename: `${title}_Runbook.docx`.replace(/[^a-z0-9._-]/gi, '_')
};
}


// ── (7) User Story Technical Design ──────────────────────────────────────
async function generateUserStoryTechnicalDesign(data) {
  const { Document, Packer, Table, TableRow, WidthType } = getDocx();
  const { C, para, h1, h2, bullets, hdrCell, dataCell, statusCell } =
makeDocHelpers();

  const storyId = safeStr(data.story_id || 'STORY');
  const title = safeStr(data.title || 'Technical Design');
  const createdDate = safeStr(data.created_date || new
Date().toLocaleDateString());
  const author = safeStr(data.author || 'Author');

  const dm = data.data_model_changes || [];
  const comps = data.implementation_components || [];
  const sec = data.security || [];
  const ints = data.integrations || [];
  const tests = (data.testing && data.testing.test_cases) ?
data.testing.test_cases : [];
  const risks = data.risks || [];

  const children = [
    h1(`${storyId} - Technical Design`),
    para(title, true, C.NAVY, 22),
    para(`Created: ${createdDate}`, false, C.GRAY, 18),
    para(`Author: ${author}`, false, C.GRAY, 18),
    para(''),

    h2('Summary'),
    para(data.summary || '')
  ];

  children.push(para(''), h2('Scope'));
  children.push(para('In Scope:', true), ...bullets(data.in_scope || []));
  children.push(para('Out of Scope:', true), ...bullets(data.out_of_scope || []));

  if (Array.isArray(data.assumptions) && data.assumptions.length) {
    children.push(para(''), h2('Assumptions'), ...bullets(data.assumptions));
  }

  if (Array.isArray(data.acceptance_criteria) && data.acceptance_criteria.length)
{
    children.push(para(''), h2('Acceptance Criteria'),
...bullets(data.acceptance_criteria));
  }

  children.push(para(''), h2('Design Overview'));
  children.push(para((data.design && data.design.overview) || ''));
```

```
  if (dm.length) {
    children.push(para(''), h2('Data Model Changes'));
    const dmTable = new Table({
      width: { size: 9360, type: WidthType.DXA },
      columnWidths: [2800, 1800, 4760],
      rows: [
        new TableRow({ children: [hdrCell('Table', 2800), hdrCell('Change', 1800),
hdrCell('Details', 4760)] }),
        ...dm.map((x, i) => new TableRow({
          children: [
            dataCell(x.table || '', 2800, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(x.change || '', 1800, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(x.details || '', 4760, i % 2 ? C.LIGHT_GRAY : C.WHITE)
          ]
        }))
      ]
    });
    children.push(dmTable);
  }

  if (comps.length) {
    children.push(para(''), h2('Implementation Components'));
    const compTable = new Table({
      width: { size: 9360, type: WidthType.DXA },
      columnWidths: [1600, 2600, 5160],
      rows: [
        new TableRow({ children: [hdrCell('Type', 1600), hdrCell('Name', 2600),
hdrCell('Description', 5160)] }),
        ...comps.map((x, i) => new TableRow({
          children: [
            dataCell(x.type || '', 1600, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(x.name || '', 2600, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(x.description || '', 5160, i % 2 ? C.LIGHT_GRAY : C.WHITE)
          ]
        }))
      ]
    });
    children.push(compTable);
  }

  if (sec.length) {
    children.push(para(''), h2('Security'), ...bullets(sec.map(s =>
`${safeStr(s.item)}: ${safeStr(s.details)}`)));
  }

  if (ints.length) {
    children.push(para(''), h2('Integrations'), ...bullets(ints.map(x =>
`${safeStr(x.system)}: ${safeStr(x.details)}`)));
  }

  if (tests.length) {
    children.push(para(''), h2('Testing Plan'));
    const tcTable = new Table({
```

```
      width: { size: 9360, type: WidthType.DXA },
      columnWidths: [1200, 4200, 3960],
      rows: [
        new TableRow({ children: [hdrCell('ID', 1200), hdrCell('Scenario', 4200),
hdrCell('Expected', 3960)] }),
        ...tests.map((t, i) => new TableRow({
          children: [
            dataCell(t.id || '', 1200, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(t.scenario || '', 4200, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(t.expected || '', 3960, i % 2 ? C.LIGHT_GRAY : C.WHITE)
          ]
        }))
      ]
    });
    children.push(tcTable);
  }

  if (Array.isArray(data.deployment_notes) && data.deployment_notes.length) {
    children.push(para(''), h2('Deployment Notes'),
...bullets(data.deployment_notes));
  }

  if (risks.length) {
    children.push(para(''), h2('Risks'));
    const riskTable = new Table({
      width: { size: 9360, type: WidthType.DXA },
      columnWidths: [4200, 1400, 3760],
      rows: [
        new TableRow({ children: [hdrCell('Risk', 4200), hdrCell('Impact', 1400),
hdrCell('Mitigation', 3760)] }),
        ...risks.map((r, i) => new TableRow({
          children: [
            dataCell(r.description || '', 4200, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            statusCell(r.impact || '', 1400),
            dataCell(r.mitigation || '', 3760, i % 2 ? C.LIGHT_GRAY : C.WHITE)
          ]
        }))
      ]
    });
    children.push(riskTable);
  }

  if (Array.isArray(data.open_questions) && data.open_questions.length) {
    children.push(para(''), h2('Open Questions'),
...bullets(data.open_questions));
  }

  const doc = new Document({ sections: [{ properties: {}, children }] });
  const blob = await Packer.toBlob(doc);
  return { blob, filename: `${storyId}_Technical_Design.docx`.replace(/[^a-z0-
9._-]/gi, '_') };
}
```

If you tell me how you're triggering generation today (UI button? script include? workspace action?) I can also provide the **dispatch function** (e.g., `generateDocByTemplateId('incident_resolution', json)`) and the **template registry** so these four show up like your existing "Project Summary" template.